

Podstawowe algorytmy grafowe

Ewa Kasprzak

29 października 2024

Opis modelu

Celem tego sprawozdania jest zbadanie i zaimplementowanie algorytmów przeszukiwania grafów, w tym przeszukiwania w głąb (DFS), przeszukiwania wszerz (BFS), sortowania topologicznego, wykrywania silnie spójnych składowych (SCC) oraz sprawdzania, czy graf jest dwudzielny. Przeszukiwanie grafów jest kluczowym zagadnieniem w teorii grafów, które ma zastosowanie w wielu dziedzinach, takich jak analiza sieci, optymalizacja tras oraz wizualizacja danych.

Do reprezentacji grafu wykorzystana zostanie lista sąsiedztwa. Wybór tej struktury danych jest uzasadniony jej efektywnością. Dla grafu z $|V|$ wierzchołkami i $|E|$ krawędziami, operacje takie jak przeszukiwanie wierzchołków i krawędzi mogą być zrealizowane w złożoności czasowej $O(|E| + |V|)$. Ta złożoność wynika z faktu, że w trakcie przeszukiwania musimy odwiedzić wszystkie wierzchołki i krawędzie. Alternatywnie, użycie macierzy sąsiedztwa nie spełniałoby tej złożoności, gdyż w przypadku grafów rzadkich (gdzie $|E| \ll |V|^2$) operacje na macierzy prowadziłyby do złożoności $O(|V|^2)$, co jest znacznie mniej efektywne.

Do implementacji wyżej wymienionych zagadnień użyto języka programowania Julia. Decyzja o wyborze tego języka podyktowana była jego zaletami w kontekście obliczeń numerycznych oraz efektywności algorytmicznej. Julia oferuje łatwą w użyciu składnię, co sprzyja szybkiemu rozwojowi i prototypowaniu algorytmów. Dodatkowo, Julia charakteryzuje się wysoką wydajnością zbliżoną do języków kompilowanych, takich jak C, co jest szczególnie ważne przy analizie dużych grafów. Wbudowane typy danych, takie jak `Vector` i `Tuple`, umożliwiają efektywne przechowywanie i manipulację danymi, co dodatkowo zwiększa wydajność algorytmów przeszukiwania.

Przeszukiwanie w głąb (DFS)

Opis Algorytmu

Algorytm przeszukiwania w głąb (DFS) eksploruje jak najgłębiej każdą gałąź, przeszukując wszystkie sąsiednie wierzchołki przed powrotem do wierzchołka nadrzędnego. W implementacji tego algorytmu wprowadzono wersję iteracyjną, aby uniknąć przepełnienia stosu, co występuje w przypadku głębokich grafów oraz implementacji rekurencyjnej.

Funkcje

1. **explore_iterative:** Wersja iteracyjna funkcji `explore`, która używa lokalnego stosu, by przechowywać wierzchołki do odwiedzenia. Ta implementacja pozwala na uniknięcie problemów związanych z głębokością rekurencji.

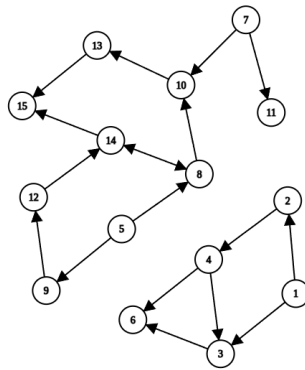
2. **dfs**: Główna funkcja, która inicjuje przeszukiwanie w głąb. Ustawia odpowiednie struktury danych do przechowywania informacji o odwiedzonych wierzchołkach i indeksach (previsit, postvisit). Funkcja ta najpierw oznacza bieżący wierzchołek jako odwiedzony, a następnie rekurencyjnie wywołuje się dla wszystkich sąsiednich wierzchołków, które nie zostały jeszcze odwiedzione.

Wyniki

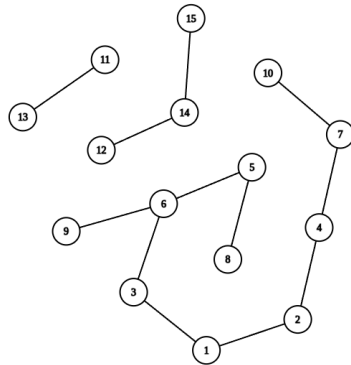
- Indeksy wierzchołków odwiedzonych podczas przeszukiwania (previsit i postvisit).
- Drzewo przeszukiwania (relacje rodzic - dziecko), które ilustruje hierarchię odwiedzonych wierzchołków.

Podsumowanie wyników testów algorytmu DFS

Algorytm DFS został przetestowany na różnych grafach skierowanych oraz nieskierowanych. Przykładem grafów, na których testowany był algorytm są:



Rysunek 1: Graf skierowany



Rysunek 2: Graf nieskierowany

Wyniki testów przedstawiono w Tabeli ???. Z danych wynika, że algorytm wykazuje dobrą skalowalność, a czas działania wzrasta w miarę zwiększania liczby wierzchołków i krawędzi.

Wierzchołki	Krawędzie	Czas działania (s)	Graf skierowany
6	9	1.9073e-6	Tak
6	9	2.8610e-6	Nie
8	12	2.8610e-6	Tak
8	12	3.0994e-6	Nie
9	17	6.9141e-6	Tak
9	17	8.1062e-6	Nie
15	12	8.1062e-6	Nie
15	18	8.8215e-6	Tak
16	33	5.0068e-6	Tak
100	261	2.0981e-5	Tak
100	262	2.0981e-5	Tak
1600	4641	0.0002210	Tak
1600	4642	0.0002220	Tak
10000	29601	0.0017431	Tak
10000	29602	0.0016408	Tak
160000	478401	0.1044550	Tak
160000	478402	0.0970600	Tak
1000000	2996001	0.6480231	Tak
1000000	2996002	0.6463358	Tak

Tabela 1: Wyniki testów algorytmu DFS

Zauważono kilka istotnych kwestii:

- **Skalowalność:** Czas działania algorytmu DFS wzrasta w miarę zwiększania liczby wierzchołków i krawędzi, co sugeruje dobrą skalowalność. Dla testów z grafami o milionie wierzchołków czas działania wyniósł około 0.65 sekundy.

- **Różnice między grafami skierowanymi i nieskierowanymi:** Nie stwierdzono istotnych różnic w czasach działania między grafami skierowanymi a nieskierowanymi przy małych rozmiarach danych wejściowych.
- **Wzrost czasów działania:** Zauważalny wzrost czasów działania dla większych grafów, co wskazuje na efektywność algorytmu w analizie dużych zbiorów danych.
- **Efektywność algorytmu:** Czas działania algorytmu jest na ogół bardzo krótki (rzędu mikrosekund dla małych grafów), co czyni go odpowiednim wyborem do przetwarzania grafów.
- **Stabilność wyników:** Czas działania wykazuje stabilność, co jest pozytywnym sygnałem dla dalszego stosowania algorytmu w praktycznych zastosowaniach.

Przeszukiwanie wszerz (BFS)

Opis Algorytmu: Algorytm przeszukiwania wszerz (BFS) eksploruje sąsiednie wierzchołki w kolejności ich odległości od wierzchołka startowego. Zaczyna od wierzchołka źródłowego, odwiedzając wszystkie sąsiadujące wierzchołki przed przejściem do ich sąsiadów.

Funkcje:

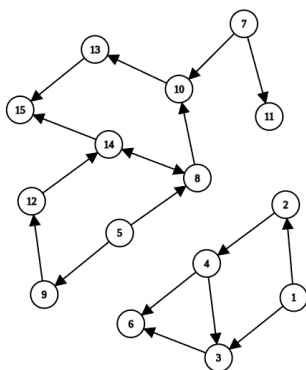
- **bfs:** Funkcja realizująca algorytm BFS. Utrzymuje kolejkę do zarządzania wierzchołkami do odwiedzenia oraz tablicę do śledzenia odległości od wierzchołka startowego. Dodatkowo, zbiera dane o previsit i postvisit.

Wyniki:

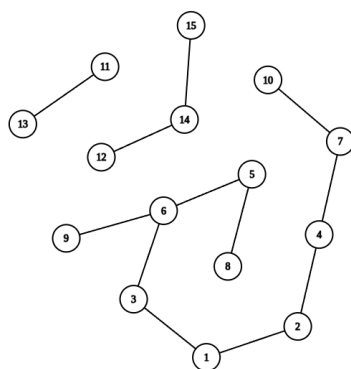
- Odległości od wierzchołka startowego do wszystkich innych wierzchołków.
- Indeksy wierzchołków odwiedzonych podczas przeszukiwania (previsit i postvisit).
- Drzewo przeszukiwania (relacja rodzic - dziecko), które ukazuje, jak wierzchołki były odkrywane.

Podsumowanie wyników testów BFS

Przykładem grafów, na których testowany był algorytm są:



Rysunek 3: Graf skierowany



Rysunek 4: Graf nieskierowany

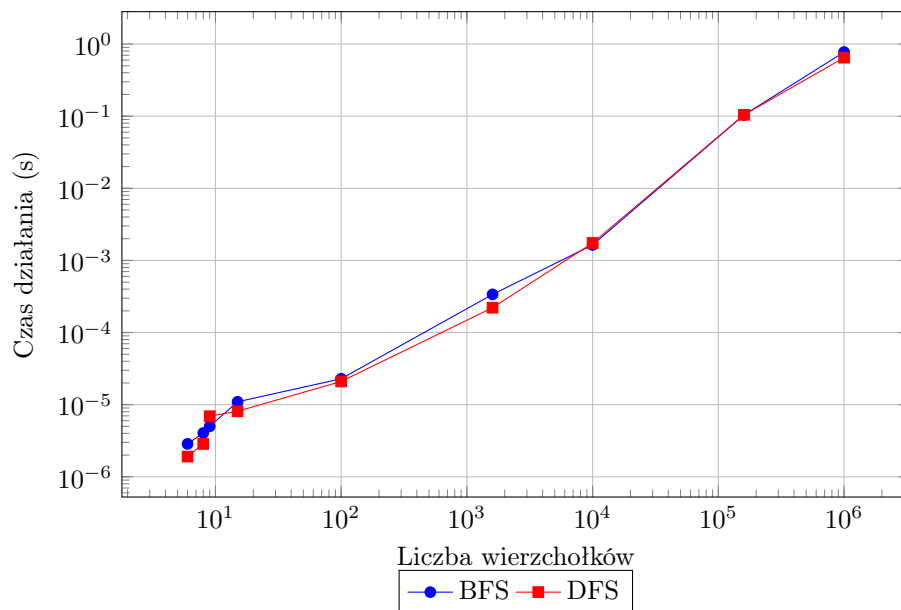
Wyniki testów dla algorytmu przeszukiwania wszerz (BFS) zostały przedstawione w Tabeli 2.

Wierzchołki	Krawędzie	Czas działania (s)	Graf skierowany
6	9	2.8610×10^{-6}	Tak
6	9	2.8610×10^{-6}	Nie
8	12	4.0531×10^{-6}	Tak
8	12	6.9141×10^{-6}	Nie
9	17	5.0068×10^{-6}	Tak
9	17	8.1062×10^{-6}	Nie
15	12	1.0967×10^{-5}	Nie
15	18	5.0068×10^{-6}	Tak
16	33	5.0068×10^{-6}	Tak
16	34	5.9605×10^{-6}	Tak
100	261	2.2888×10^{-5}	Tak
100	262	2.5988×10^{-5}	Tak
1600	4641	0.0003381	Tak
1600	4642	0.0002282	Tak
10000	29601	0.0016410	Tak
10000	29602	0.0016570	Tak
160000	478401	0.1042230	Tak
160000	478402	0.1002421	Tak
1000000	2996001	0.7737298	Tak
1000000	2996002	0.7814999	Tak

Tabela 2: Wyniki testów algorytmu BFS

Wnioski z wyników BFS

- **Czas działania** algorytmu BFS wzrasta w miarę zwiększania się liczby wierzchołków i krawędzi, co jest zgodne z oczekiwaniami, ponieważ algorytm przeszukuje wszystkie sąsiadujące wierzchołki przed przejściem do głębszych poziomów.
- Czas działania BFS dla grafów skierowanych i nieskierowanych jest porównywalny w przypadku niewielkich rozmiarów grafów, ale przy większych rozmiarach (np. powyżej 10000 wierzchołków) zauważalne są różnice w czasie działania.
- **Porównanie z DFS:** W przypadku mniejszych grafów (do 1000 wierzchołków) czasy działania dla BFS i DFS są porównywalne. Natomiast dla większych grafów BFS ma tendencję do zajmowania więcej czasu niż DFS, co może być spowodowane potrzebą zarządzania kolejką.
- **Wydajność przy dużych danych:** Dla grafów o rozmiarze 1,000,000 wierzchołków BFS osiąga czas 0.7737 s, co jest znacznie wolniejsze w porównaniu do DFS, który osiągał wyniki na poziomie 0.6480 s dla podobnych rozmiarów grafów.



Rysunek 5: Porównanie czasu działania algorytmu DFS i BFS

Sortowanie topologiczne

Opis Algorytmu

Sortowanie topologiczne jest techniką stosowaną w grafach skierowanych, która porządkuje wierzchołki w taki sposób, że dla każdej krawędzi (u, v) , wierzchołek u pojawia się przed wierzchołkiem v . Algorytm sprawdza również, czy graf zawiera cykle, ponieważ sortowanie topologiczne jest możliwe tylko dla grafów acyklicznych.

Funkcje

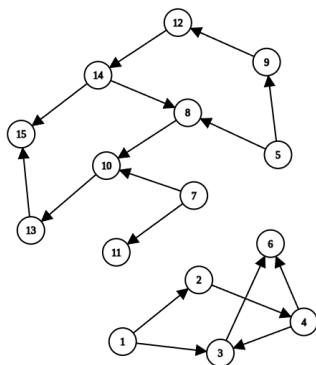
- **topological_sort**: Wykorzystuje algorytm przeszukiwania w głąb (DFS) do generowania porządku topologicznego. Sprawdza również cykle w grafie. Jeśli graf zawiera cykle, zwraca odpowiedni komunikat.
- **is_cyclic**: Funkcja ta analizuje graf w celu wykrycia cykli. Używa metody DFS z dodatkowymi znacznikami (np. odwiedzionymi wierzchołkami) do identyfikacji wierzchołków, które są aktualnie w procesie przeszukiwania. Jeśli napotka wierzchołek, który jest już w trakcie przeszukiwania, oznacza to, że istnieje cykl.

Wyniki

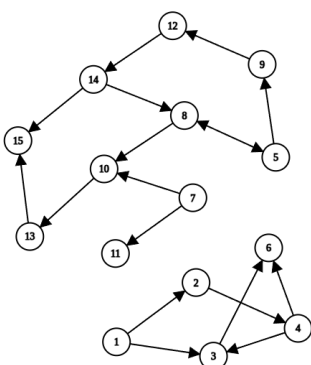
- Porządek topologiczny wierzchołków, jeśli graf jest acykliczny.
- Informacje o istnieniu cyklu w grafie, co może uniemożliwić utworzenie poprawnego porządku topologicznego.

Podsumowanie wyników testów sortowania topologicznego

Przykładem grafów, które zostały użyte do testowania algorytmu są:



Rysunek 6: Graf skierowany acykliczny



Rysunek 7: Graf ze skierowanym cyklem

Wyniki testów dla algorytmu sortowania topologicznego zostały przedstawione w Tabeli 3.

Wierzchołki	Krawędzie	Czas działania (sekundy)
16	33	7.1526e-6
16	34	0.0060940
100	261	2.6941e-5
100	262	0.0056422
1600	4641	0.0002570
1600	4642	0.0067511
10000	29601	0.0023599
10000	29602	0.0076852
160000	478401	0.1298158
160000	478402	0.1268361
1000000	2996001	0.9502229
1000000	2996002	0.7828231

Tabela 3: Czasy działania algorytmu sortowania topologicznego

Wnioski

- **Zależność czasu od liczby wierzchołków i krawędzi:** Czas działania algorytmu sortowania topologicznego wzrasta wraz z liczbą wierzchołków i krawędzi w grafie. Zauważono, że dla dużych grafów acyklicznych, czasy działania rosną znacząco, co jest naturalnym efektem większej liczby operacji przeszukiwania i sortowania.
- **Efektywność algorytmu:** Algorytm sortowania topologicznego jest skutecznym narzędziem do analizy grafów acyklicznych. W praktycznych zastosowaniach ważne jest, aby brać pod uwagę cykle w grafach, które uniemożliwiają wykonanie poprawnego porządku topologicznego.
- **Zastosowania w rzeczywistości:** Wyniki sugerują, że algorytm jest odpowiedni do analizy dużych grafów acyklicznych, ale w przypadku grafów z cyklami może być konieczne zastosowanie alternatywnych strategii analizy.

Silnie Spójne Składowe (SCC)

Opis Algorytmu: Algorytm do wykrywania silnie spójnych składowych w grafie skierowanym dzieli graf na podgrafy, w których każdy wierzchołek jest osiągalny z każdego innego wierzchołka w tej samej składowej. Wykorzystuje przeszukiwanie DFS na odwróconym grafie wejściowym oraz kolejność wartości `postvisit` dla tego grafu.

Funkcje

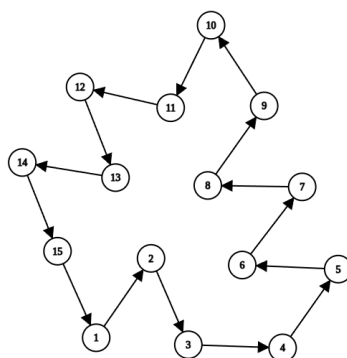
- `scc`: Wykonuje DFS na odwróconym grafie i zbiera silnie spójne składowe. Zwraca listę tych składowych oraz ich rozmiary.

Wyniki

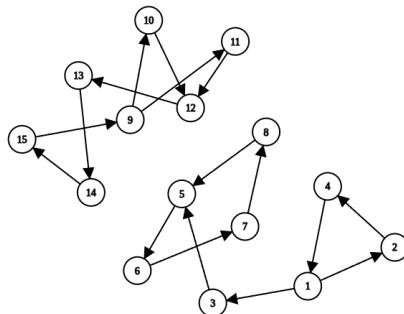
- Liczba silnie spójnych składowych.
- Rozmiary poszczególnych składowych.
- Wierzchołki należące do każdej składowej, jeśli ich liczba jest mniejsza lub równa 200.

Podsumowanie wyników testów SCC

Przykładem grafów, na których testowany był algorytm są:



Rysunek 8: Graf silnie spójny



Rysunek 9: Graf ze spójnymi składowymi

Wyniki testów dla algorytmu SCC zostały przedstawione w Tabeli 4.

Wierzchołki	Krawędzie	Czas działania (s)
15	17	8.106231689453125e-6
15	15	8.821487426757812e-6
16	39	9.775161743164062e-6
107	185	3.600120544433594e-5
1008	1609	0.0002598762512207031
10009	15943	0.0029091835021972656
100010	159679	0.10667800903320312
1000011	1598897	0.7704451084136963

Tabela 4: Wyniki działania algorytmu SCC na różnych grafach.

Wnioski

- Czas działania algorytmu SCC rośnie wraz z liczbą wierzchołków i krawędzi w grafie, co jest oczekiwanym wynikiem, gdyż algorytm wymaga przeszukiwania całego grafu.
- W miarę wzrostu liczby wierzchołków i krawędzi, czas działania algorytmu wzrasta znacznie, osiągając około 0.77 s dla grafu z ponad 1 milionem wierzchołków i prawie 1.6 miliona krawędzi, co wskazuje na skalowalność algorytmu, ale też na jego rosnącą złożoność obliczeniową.
- Wyniki sugerują, że algorytm SCC jest efektywny dla małych i średnich grafów, jednak dla bardzo dużych grafów czas działania może stać się znaczący i wymagać optymalizacji.

Wykrywanie grafu dwudzielnego

Opis Algorytmu: Algorytm do wykrywania grafu dwudzielnego sprawdza, czy możliwe jest podzielenie wierzchołków grafu na dwa zbiory, w których każda krawędź łączy wierzchołki z różnych zbiorów. Używa przeszukiwania wszerz (BFS) do przypisania kolorów wierzchołkom.

Funkcje

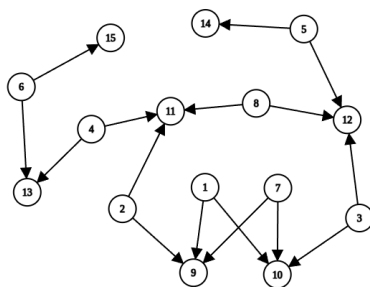
- `is_bipartite`: Sprawdza, czy graf jest dwudzielny, kolorując wierzchołki w trakcie przeszukiwania.

Wyniki

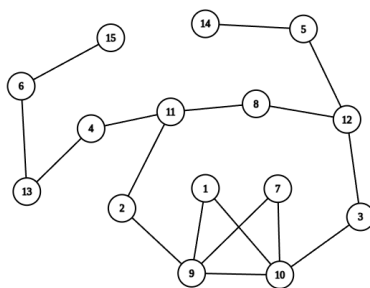
- Informacja, czy graf jest dwudzielny.
- Zbiory wierzchołków, jeśli graf jest dwudzielny.

Podsumowanie wyników testów dwudzielności grafów

Przykładem grafów, na których testowany był algorytm są:



Rysunek 10: Graf skierowany dwudzielny



Rysunek 11: Graf nieskierowany niedwudzielny

Wyniki testów dla algorytmu wykrywania dwudzielności grafu zostały przedstawione w Tabeli 5.

Liczba wierzchołków	Liczba krawędzi	Czas działania (s)	Czy dwudzielny
15	16	0.0	true
15	17	1.1920928955078125e-6	false
15	22	4.0531158447265625e-6	true
15	22	4.0531158447265625e-6	false
16	24	2.1457672119140625e-6	true
16	25	9.5367431640625e-7	false
100	180	6.9141387939453125e-6	true
100	181	6.9141387939453125e-6	false
10000	19800	0.00023889541625976562	true
10000	19801	0.00010704994201660156	false
1600	3120	2.5987625122070312e-5	true
1600	3121	1.5974044799804688e-5	false
160000	319200	0.012763023376464844	true
160000	319201	0.00664210319519043	false
1000000	1998000	0.1014871597290039	true
1000000	1998001	0.04789304733276367	false
131071	196606	0.00488591194152832	true
131071	196606	0.018544912338256836	false
16383	24574	0.0004677772521972656	true
16383	24574	0.0003178119659423828	false

Tabela 5: Wyniki eksperymentu dla algorytmu is bipartite

Wnioski

- Czas działania algorytmu rośnie wraz z liczbą wierzchołków i krawędzi w grafie, co jest oczekiwanym wynikiem, ponieważ algorytm wymaga przeszukiwania całego grafu.
- Czas działania algorytmu wzrasta znacznie w miarę zwiększania liczby wierzchołków i krawędzi, osiągając około 0.1 s dla grafu z ponad 1 milionem wierzchołków i prawie 2 milionami krawędzi, co potwierdza skalowalność algorytmu, ale również rosnącą złożoność obliczeniową.
- Wyniki sugerują, że zaimplementowany algorytm ‘wykrywania dwudzielności grafu’ jest efektywny dla małych i średnich grafów, jednak dla bardzo dużych grafów czas działania może stać się znaczący i wymagać optymalizacji.